

Pipeline for basic z buffer

- Vertex stage (input: position / vtx; color / tri)
 - transform position (object to screen space)
 - pass through color
- Rasterizer
 - interpolated parameter: z' (screen z)
 - pass through color
- Fragment stage (output: color, z')
 - write to color planes only if interpolated $z' < \text{current } z'$

Pipeline for Gouraud shading

- Vertex stage (input: position, color, and normal / vtx)
 - transform position and normal (object to eye space)
 - compute shaded color per vertex
 - transform position (eye to screen space)
- Rasterizer
 - interpolated parameters: z' (screen z); r, g, b color
- Fragment stage (output: color, z')
 - write to color planes only if interpolated $z' < \text{current } z'$

Pipeline for Phong shading

- Vertex stage (input: position, color, and normal / vtx)
 - transform position and normal (object to eye space)
 - transform position (eye to screen space)
 - pass through color
- Rasterizer
 - interpolated parameters: z' (screen z); r, g, b color; x, y, z normal
- Fragment stage (output: color, z')
 - compute shading using interpolated color and normal
 - write to color planes only if interpolated $z' < \text{current } z'$

Gouraud shading (vertex shader)

```
#version 430

in vec3 VertexPosition;
in vec3 VertexNormal;
in vec2 VertexTex;

out Data
{
    vec3 FrontColor;
    vec3 BackColor;
    vec2 TexCoord;
} data;

struct LightInfo
{
    vec3 Position;    //Light Position in eye-coords
    vec3 La;          //Ambient light intensity
    vec3 Ld;          //Diffuse light intensity
    vec3 Ls;          //Specular light intensity
};
```

```

struct MaterialInfo
{
    vec3 Ka;           //Ambient reflectivity
    vec3 Kd;           //Diffuse reflectivity
    vec3 Ks;           //Specular reflectivity
    float Shininess;    //Specular shininess factor
};

uniform LightInfo Light[LIGHTCOUNT];
uniform MaterialInfo Material;

uniform mat4 ModelViewMatrix;
uniform mat3 NormalMatrix;
uniform mat4 MVP;

void getEyeSpace( out vec3 norm, out vec3 position )
{
    norm = normalize( NormalMatrix * VertexNormal );
    position = vec3( ModelViewMatrix * vec4( VertexPosition, 1 ) );
}

```

```

vec3 light( int lightIndex, vec3 position, vec3 norm )
{
    vec3 s = normalize( vec3( Light[lightIndex].Position - position ) );
    vec3 v = normalize( -position.xyz );
    vec3 r = reflect( -s, norm );

    vec3 ambient = Light[lightIndex].La * Material.Ka;

    float sDotN = max( dot( s, norm ), 0.0 );
    vec3 diffuse = Light[lightIndex].Ld * Material.Kd * sDotN;

    vec3 spec = vec3( 0.0 );
    if ( sDotN > 0.0 )
        spec = Light[lightIndex].Ls * Material.Ks * pow( max( dot(r,v) , 0.0 ), Material.Shininess );

    return ambient + diffuse + spec;
}

```

```

void main()
{
    vec3 eyeNorm;
    vec3 eyePosition;
    getEyeSpace( eyeNorm, eyePosition );

    data.FrontColor = vec3(0);
    data.BackColor = vec3(0);

    for( int i=0; i<LIGHTCOUNT; ++i )
    {
        data.FrontColor += light( i, eyePosition, eyeNorm );
        data.BackColor += light( i, eyePosition, -eyeNorm );
    }

    data.TexCoord = VertexTex;
    gl_Position = MVP * vec4( VertexPosition, 1 );
}

```

Gouraud shading (fragment shader)

```
#version 430

in Data
{
    vec3 FrontColor;
    vec3 BackColor;
    vec2 TexCoord;
} data;

out vec4 FragColor;

uniform sampler2D Tex;

void main()
{
    if ( gl_FrontFacing )
        FragColor = vec4(data.FrontColor, 1);
    else
        FragColor = vec4(data.BackColor, 1);

    FragColor *= texture(Tex, data.TexCoord);
}
```

Phong shading (vertex shader)

```
#version 430

in vec3 VertexPosition;
in vec3 VertexNormal;
in vec2 VertexTex;

out Data
{
    vec3 Position;
    vec3 Normal;
    vec2 TexCoord;
} data;

uniform mat4 ModelViewMatrix;
uniform mat3 NormalMatrix;
uniform mat4 MVP;

void main()
{
    data.Normal = normalize( NormalMatrix * VertexNormal );
    data.Position = vec3( ModelViewMatrix * vec4( VertexPosition, 1 ) );
    data.TexCoord = VertexTex;

    gl_Position = MVP * vec4( VertexPosition, 1 );
}
```

Phone shading (fragment shader)

```
#version 430

struct LightInfo
{
    vec3 Position;    //Light Position in eye-coords
    vec3 La;          //Ambient light intensity
    vec3 Ld;          //Diffuse light intensity
    vec3 Ls;          //Specular light intensity
};

struct MaterialInfo
{
    vec3 Ka;           //Ambient reflectivity
    vec3 Kd;           //Diffuse reflectivity
    vec3 Ks;           //Specular reflectivity
    float Shininess;   //Specular shininess factor
};

in Data
{
    vec3 Position;
    vec3 Normal;
    vec2 TexCoord;
} data;

out vec4 FragColor;

uniform LightInfo Light[LIGHTCOUNT];
uniform MaterialInfo Material;
uniform sampler2D Tex;
```

```
void light( int lightIndex, vec3 position, vec3 norm, out vec3 ambient, out vec3 diffuse, out vec3 spec )
{
    vec3 n = normalize( norm );
    vec3 s = normalize( Light[lightIndex].Position - position );
    vec3 v = normalize( -position );
    vec3 r = reflect( -s, n );

    ambient = Light[lightIndex].La * Material.Ka;

    float sDotN = max( dot( s, n ), 0.0 );
    diffuse = Light[lightIndex].Ld * Material.Kd * sDotN;

    spec = Light[lightIndex].Ls * Material.Ks * pow( max( dot(r,v) , 0.0 ), Material.Shininess );
}
```

```

void main()
{

    vec3 ambientSum = vec3(0);
    vec3 diffuseSum = vec3(0);
    vec3 specSum = vec3(0);
    vec3 ambient, diffuse, spec;

    if ( gl_FrontFacing )
    {
        for( int i=0; i<LIGHTCOUNT; ++i )
        {
            light( i, data.Position, data.Normal, ambient, diffuse, spec );
            ambientSum += ambient;
            diffuseSum += diffuse;
            specSum += spec;
        }
    }
    else
    {
        for( int i=0; i<LIGHTCOUNT; ++i )
        {
            light( i, data.Position, -data.Normal, ambient, diffuse, spec );
            ambientSum += ambient;
            diffuseSum += diffuse;
            specSum += spec;
        }
    }
    ambientSum /= LIGHTCOUNT;

    vec4 texColor = texture(Tex, data.TexCoord);

    FragColor = vec4( ambientSum + diffuseSum, 1 ) * texColor + vec4( specSum, 1 );
}

```